

HƯỚNG DẪN SỬ DỤNG GÓI `ex_test`

Tài liệu dùng cho mã nguồn `ex_test.typ`.

Phiên bản gói: 1.0.0 — Tác giả: Phan Văn Anh

Github: <https://github.com/vananh2801>

LỜI CẢM ƠN

Tôi chân thành cảm ơn thầy Trần Anh Tuấn và thầy Dương Phước Sang vì đã tạo ra gói `ex_test` rất tuyệt vời, giúp việc gõ LaTeX trở nên gần gũi hơn cho GV trong dạy học.

Dựa trên cơ sở ý tưởng của `ex_test`, tôi đã làm gói lệnh này, phục vụ cho việc gõ Typst trong dạy học.

MỤC LỤC

LỜI CẢM ƠN	3
Chương I. GIỚI THIỆU	7
1. <code>project</code> – cấu trúc thư mục kiến nghị	8
2. <code>main.typ</code> – khởi tạo file chính	9
3. <code>heading</code> – các cấp tiêu đề	10
Chương II. TỰ TẠO THEOREM	11
1. <code>#createTheoremEx</code> – tạo một theorem mới	12
2. <code>#immini</code> <code>#imminiL</code> – hình minh hoạ cho bài tập	15
3. <code>#loigiaiai</code> – lời giải	16
4. <code>#chc</code> – câu hỏi con	18
Chương III. TRẮC NGHIỆM VÀ TỰ LUẬN	21
1. <code>#choice</code> – trắc nghiệm 4 phương án	22
2. <code>#choiceTF</code> – trắc nghiệm Đúng/Sai	24
3. <code>#shortans</code> – câu trả lời ngắn	26
4. <code>#BatDauLuuDapAn</code> <code>#KetThucLuuDapAn</code> <code>#XuatDapAn</code> – lưu và xuất bảng đáp án tự động ...	27
5. <code>#dapso</code> – đáp số vắn tắt cho bài toán tự luận	28
6. <code>#listEX</code> <code>#itemize</code> <code>#enumerate</code> – tạo danh sách	29

Chương I

GIỚI THIỆU

PROJECT – CẤU TRÚC THƯ MỤC KIẾN NGHỊ

Dự án mẫu có cấu trúc như sau:

```
project/
├── main.typ
├── style/
│   ├── ex_test.typ
│   └── lib.typ
├── data/
│   └── 9D1-1.typ
└── imgs/
```

Trong đó:

- `main.typ` là file chính của dự án dùng để khai báo `#import` và `#include`.
- `ex_test.typ` là phần cốt lõi của `ex_test`, định nghĩa cách một “theorem” (định nghĩa/ví dụ/bài tập...) hoạt động, cách trích nghiệm lưu — xuất đáp án, cách danh sách tự chia cột, cách hình minh hoạ ghép với văn bản...
Thầy cô muốn có thể sửa để thay đổi theo nhu cầu của thầy cô, nhưng thường thì không cần thay đổi.
- `lib.typ` file dùng để cấu hình trang giấy, heading, footer,... Thầy cô có thể thay đổi theo sở thích cá nhân. Ngoài ra, trong file này còn có các khai báo như tạo theorem, cách các theorem hiển thị.
- File `9D1-1.typ` là file dữ liệu thể hiện cách mà các theorem đã khai báo được sử dụng.

Lưu ý: các lệnh của `ex_test` dùng trong file dữ liệu hầu như có tham số là tiếng Việt, còn các lệnh dùng để khai báo hầu như có tham số là tiếng Anh. Mục đích là giúp tránh nhầm lẫn trong quá trình soạn thảo và quá trình bảo trì gói `ex_test`.

MAIN.TYP – KHỞI TẠO FILE CHÍNH

Ở đầu các file đuôi `.typ`, thầy cô cần khai báo:

```
#import "style/ex_test.typ"
```

Thật ra, nếu thầy cô xem kĩ các file mẫu sẽ thấy được rằng nếu trong `lib.typ` mà đã có dòng lệnh này thì khi thầy cô chạy lệnh `#import "style/lib.typ"` ở đầu các file thì đã đi kèm `ex_test` rồi.

HEADING – CÁC CẤP TIÊU ĐỀ

Gói `ex_test.typ` định nghĩa lại các cấp tiêu đề LaTeX quen thuộc như:

Lệnh	Cấp tương đương bên Typst
<code>#chapter[...]</code>	<code>=</code>
<code>#section[...]</code>	<code>==</code>
<code>#subsection[...]</code>	<code>===</code>
<code>#subsubsection[...]</code>	<code>====</code>
<code>#paragraph[...]</code>	<code>=====</code>
<code>#subparagraph[...]</code>	<code>=====</code>

Ví dụ minh họa:

```
#chapter[Phương trình bậc nhất hai ẩn]
#section[Khái niệm phương trình bậc nhất hai ẩn]
#subsection[Lý thuyết]
#subsubsection[Phương trình bậc nhất hai ẩn]
```

Chương II

TỰ TẠO THEOREM

#CREATETHEOREMEX – TẠO MỘT THEOREM MỚI

A Lệnh khai báo đầy đủ

Để tạo một `theorem` mới để có thể dùng các chức năng của `ex_test`, thầy cô dùng lệnh `#createTheoremEx` như sau:

```
#let <ten> = createTheoremEx(
  "<ten>",           // tên lệnh, đặt trong ""
  "<nhan>",          // nhãn hiển thị, đặt trong ""
  boxF: <ham-khung>, // mặc định: không khung
  contentF: <ham-noidung>, // mặc định: in đậm nhãn + nội dung
  imminiF: <ham-nhan-immini>, // nhãn dùng khi có #immini
  numbered: <true/false>, // false = không đánh số
)
```

B Khai báo cơ bản

Ví dụ minh họa khai báo cơ bản:

```
#let minhhoa = createTheoremEx(
  "minhhhoa",
  "Minh hoạ",
)
```

Ví dụ minh họa sử dụng:

```
#minhhhoa(tieude: "Tên minh hoạ")[
  Đây là phần minh hoạ của cách sử dụng.
]
```

Kết quả trên PDF:

Minh hoạ 1 (Tên minh hoạ). Đây là phần minh hoạ của cách sử dụng.

C Bộ đếm số

Khi sang Chương mới hoặc Bài mới, các bộ đếm `vd`, `bt`, `btrl`, `ex` ... sẽ KHÔNG tự động reset về 0. Tuy nhiên gói lệnh `ex_test` đã tạo sẵn các counter. Nếu thầy cô muốn reset về 0 thì thầy cô phải chèn lệnh sau vào phần cấu hình heading:

```
#counter("<tên theorem>_count").update(0)
```

Ví dụ minh họa:

```
#counter("ex_count").update(0)
#counter("bt_count").update(0)
#counter("vd_count").update(0)
```

Thầy cô có thể tham khảo `show heading.where(level: 1)` trong `lib.typ`.

D boxF – tạo khung

Khai báo khung cho Minh họa:

```
#let minhhoaBoxF(body) = {           // tên tùy ý
  block(
    width: 100%,                     // bề ngang hiện đủ 100% trang giấy
    radius: 6pt,                     // độ bo tròn
    inset: 1em,                      // khoảng cách giữa viền và nội dung
    fill: blue.lighten(90%),         // màu nền
    stroke: 1pt + blue,              // độ dày và màu viền
    breakable: true,                // cho phép ngắt giữa trang
  )[#body]
}
```

Khi đó ta khai báo như sau:

```
#let minhhoa = createTheoremEx(
  "minhhoa",
  "Minh họa",
  boxF: minhhoaBoxF,
)
```

Kết quả khi sử dụng:

Minh họa 2 (Tên minh họa). Đây là phần minh họa của cách sử dụng.

E contentF imminiF – thay đổi cấu trúc của nội dung

Đôi khi ta muốn đưa nguồn bài tập xuống cạnh dưới bên phải khung thay vì in đậm ở phần nhãn, ta cần khai báo lại `imminiF` và `contentF`, chẳng hạn sau đây là khai báo cho Minh họa:

```
#let minhhoaContentF(body, fulllabel: none, shortlabel: none, label: none, title:
none, count: none) = [
  // In đậm Tiêu đề
  #strong[#shortlabel]
  // Xuất nội dung
  #body
  // Xuất nguồn đề bài in nghiêng
  #if title != none {
    [#parbreak()#h(1fr)#emph[Nguồn: #title]]
  }
]
```

```
#let minhhoaImminiF(fulllabel: none, shortlabel: none, label: none, title: none,
count: none) = [
  #strong[#shortlabel]
]
```

Khi đó ta khai báo như sau:

```
#let minhhoa = createTheoremEx(
  "minhhhoa",
  "Minh hoạ",
  boxF: minhhoaBoxF,
  contentF: minhhoaContentF,
  imminiF: minhhoaImminiF
)
```

Kết quả khi sử dụng:

Minh hoạ 3. Đây là phần minh hoạ của cách sử dụng.

Nguồn: Tên minh hoạ

Lưu ý: Để rõ hơn thầy cô xem ở cuối file `lib.typ`.

#IMMINI #IMMINIL – HÌNH MINH HOẠ CHO BÀI TẬP

Dùng lệnh `#immini` để xếp đề bài bên trái, hình bên phải. Ngược lại, dùng `#imminiL` để xếp hình bên trái, đề bài bên phải.

Ví dụ minh họa:

```
#luuy[
  #immini[
    Toạ độ giao điểm của hai đường thẳng chính là nghiệm của hệ phương
    trình tương ứng.
  ]
]

#luuy[
  #imminiL[
    Toạ độ giao điểm của hai đường thẳng chính là nghiệm của hệ phương
    trình tương ứng.
  ]
]
```

Kết quả:

Lưu ý. Toạ độ giao điểm của hai đường thẳng chính là nghiệm của hệ phương trình tương ứng.

(hình minh họa ở đây)

(hình minh họa ở đây)

Lưu ý. Toạ độ giao điểm của hai đường thẳng chính là nghiệm của hệ phương trình tương ứng.

#LOIGIAI – LỜI GIẢI

`#loigiai[...]` đặt bên trong một theorem (`vd` , `bt` , `btrl` , `ex` , `chc` ,...) để chứa lời giải. Lệnh này **tự động** kiểm tra cấu hình `#state("<ten>_show_ans")` để quyết định hiển thị lời giải hay thay bằng dòng kẻ chấm (nếu đã dùng `#dotlinefull`).

`#dotlineEX(n)` : in ra trực tiếp `n` dòng kẻ chấm (dùng ngay trong `#loigiai`).

`#dotlinefull("bt", socot: 2)` : đặt ở đầu file, khiến **mọi** `#loigiai` của `bt` tự động thay bằng dòng kẻ chấm với số dòng được tính tự động theo độ dài lời giải gốc, chia thành `socot` cột.

Ví dụ minh hoạ 1:

```
#bt[
  Đề bài...
  #loigiai[
    Hướng dẫn giải...
  ]
]
```

Kết quả:

Bài 1. Đề bài...

Lời giải.

Hướng dẫn giải...

Ví dụ minh hoạ 2:

```
#dotlineEX(5, socot: 2)
```

Kết quả:

.....	
.....	
.....	
.....	
.....	

Lưu ý: Để đổi tiêu đề của #loigiai , thầy cô khai báo:

```
#loigiaiEX.update([*Hướng dẫn giải.*])
```

#CHC – CÂU HỎI CON

Đối với các dạng đề như thi Đánh giá năng lực, thường sẽ gặp trường hợp có nhiều câu hỏi dùng chung giả thiết. Khi đó ta dùng `#chc` ngay sau `theorem` chứa giả thiết chung.

A Cú pháp

```
#<tên theorem>(sochc: <số câu hỏi con>)[
  Giả thiết chung (ngữ cảnh)...
]

#chc(tieude: <nguồn riêng câu này, có thể bỏ trống>)[
  Nội dung câu hỏi con thứ nhất...
  #loigiaihai[lời giải riêng của câu này...]
]

#chc[
  Nội dung câu hỏi con thứ hai...
]
```

- `sochc: n` khai báo trên `theorem` : cho biết có `n` câu hỏi con bên trong. Khi đó, thay vì hiển thị “Câu 5”, tiêu đề `theorem` sẽ tự đổi thành dạng **Dựa vào thông tin dưới đây để trả lời các câu từ X đến $X + n$** . Trong đó, X là đánh số hiện tại của `theorem` .
- Nếu KHÔNG khai báo `sochc:` (hoặc để `sochc: 0`), các `#chc` bên trong sẽ tự đánh số kiểu “X.a”, “X.b”.... Nếu `theorem` không đánh số thì sẽ hiện “a”, “b”, “c”...
- Mỗi `#chc[...]` có thể được sử dụng độc lập với `theorem` , copy tất cả cấu hình áp dụng cho `theorem` , có thể dùng `#loigiaihai[...]` riêng và `title:` (nguồn) riêng cho từng câu con...

B Ví dụ minh họa

```
#minhhhoa(sochc: 2)[
    Cho hai đường thẳng $d_1: y = x + 1$ và $d_2: y = -x + 3$.
]

#chc[
    Tìm giao điểm của $d_1$ và $d_2$.
    #loigiai[
        Giải hệ $heva(y = x + 1, y = -x + 3)$ được $x = 1$, $y = 2$.
    ]
]

#chc[
    Vẽ hai đường thẳng trên cùng một hệ trục tọa độ.
    #loigiai[
        Học sinh tự vẽ dựa theo 2 điểm đặc biệt của mỗi đường thẳng.
    ]
]

#minhhhoa[
    Cho hai đường thẳng $d_1: y = x + 1$ và $d_2: y = -x + 3$.
]

#chc[
    Tìm giao điểm của $d_1$ và $d_2$.
    #loigiai[
        Giải hệ $heva(y = x + 1, y = -x + 3)$ được $x = 1$, $y = 2$.
    ]
]

#chc[
    Vẽ hai đường thẳng trên cùng một hệ trục tọa độ.
    #loigiai[
        Học sinh tự vẽ dựa theo 2 điểm đặc biệt của mỗi đường thẳng.
    ]
]
```

Kết quả:

Dựa vào thông tin dưới đây để trả lời các câu từ 4 đến 5.

Cho hai đường thẳng $d_1 : y = x + 1$ và $d_2 : y = -x + 3$.

Minh họa 4. Tìm giao điểm của d_1 và d_2 .

Lời giải.

Giải hệ hai phương trình trên, ta được $x = 1, y = 2$.

Minh hoạ 5. Vẽ hai đường thẳng trên cùng một hệ trục toạ độ.

Lời giải.

Học sinh tự vẽ dựa theo 2 điểm đặc biệt của mỗi đường thẳng.

Minh hoạ 6. Cho hai đường thẳng $d_1 : y = x + 1$ và $d_2 : y = -x + 3$.

Minh hoạ 6.a. Tìm giao điểm của d_1 và d_2 .

Lời giải.

Giải hệ $\begin{cases} y = x + 1 \\ y = -x + 3 \end{cases}$ được $x = 1, y = 2$.

Minh hoạ 6.b. Vẽ hai đường thẳng trên cùng một hệ trục toạ độ.

Lời giải.

Học sinh tự vẽ dựa theo 2 điểm đặc biệt của mỗi đường thẳng.

C Đổi chữ “Dựa vào thông tin dưới đây...”

Câu dẫn mặc định trước nhóm câu hỏi con lấy từ 2 biến `fromchc` và `tochc` :

```
#fromchc.update("Dựa vào thông tin dưới đây để trả lời các câu từ")  
#tochc.update("đến")
```

Đặt lệnh này ở `main.typ` (trước phần nội dung) nếu muốn đổi sang cách diễn đạt khác, ví dụ rút gọn hoặc dùng cho đề thi kiểu khác.

Chương III

TRẮC NGHIỆM VÀ TỰ LUẬN

#CHOICE – TRẮC NGHIỆM 4 PHƯƠNG ÁN

- Tham số `socot` : ép số cột hiển thị 4 phương án (1, 2 hoặc 4). Nếu bỏ trống, package tự tính cột tối ưu theo độ rộng nội dung.
- `dapan: "B"` : khai báo nếu muốn xuất bảng đáp án tự động bằng `#XuatDapAn` (xem bên dưới).
- Nếu muốn thay các chữ cái A, B, C, D, thành số 1, 2, 3, 4 thì dùng `#choiceN` .
- Nếu chỉ truyền 1 phương án, `#choice` sẽ vẽ dạng `underbracket` đánh số A, B, C... tăng dần theo thứ tự xuất hiện (dùng cho trắc nghiệm tìm lỗi sai của môn Anh ngữ).
- `khoangcach: 1em` : chỉnh khoảng cách giữa các dòng phương án khi các đáp án xếp thành nhiều dòng. Tham số này cũng có ở `#choiceN` và `#choiceTF` .
- Cuối mỗi phương án (`choice` / `choiceN` / `choiceTF`) đều tự in thêm ký tự lấy từ biến `dotEX` (mặc định là dấu `.`). Muốn đổi (ví dụ bỏ hẵn dấu chấm), đặt ở `main.typ` : `#dotEX.update([])` .

Ví dụ minh hoạ 1:

```
#ex[
  Phương trình nào sau đây KHÔNG là phương trình bậc nhất hai ẩn?
  #choice(
    [$x - 2y = 5$],
    [$0x + 0y = -3$],
    [$6x + 0y = 1$],
    [$0x - 4y = 3$],
    socot: 4,
    dapan: "B",
  )
  #loigiaiai[
    Đáp án B vì $a = b = 0$.
  ]
]
```

Kết quả:

Câu 1. Phương trình nào sau đây KHÔNG là phương trình bậc nhất hai ẩn?

- A. $x - 2y = 5$. B. $0x + 0y = -3$. C. $6x + 0y = 1$. D. $0x - 4y = 3$.

Lời giải.

Đáp án B vì $a = b = 0$.

Ví dụ minh hoạ 2:

```
#ex[
  Yoghurt, #choice[one of] the #choice[healthiest] snacks, #choice(dapan: "B")[are]
  adviced for people with #choice[daily] calcium needs.
]
```

Kết quả:

Câu 2. Yoghurt, one of the healthiest snacks, are adviced for people with daily calcium needs.
 A B C D

#CHOICETF – TRẮC NGHIỆM ĐÚNG/SAI

- Mặc định sẽ không kẻ bảng, thêm tham số `kieu: "t"` để kẻ bảng.
- Khi không có tham số `kieu: "t"` sẽ in dạng danh sách chia cột như `#choice`. Nếu muốn chỉnh số cột thì thêm tham số `socot: 2`.
- Lệnh `#itemchoice` cũng có thể thêm tham số `socot: 2`.
- Lệnh `#itemchoice` tự động đọc tham số `dapan` đã khai báo ở `#choiceTF` cùng câu để in đậm sẵn nhãn “a) Đúng”, “b) Sai”... ở đầu mỗi ý. Vì vậy nội dung từng dòng bên trong `#itemchoice[...]` chỉ nên đưa ra các bước giải, KHÔNG cần gõ lại chữ “Đúng”/“Sai” — nếu gõ lại sẽ bị lặp

Ví dụ minh hoạ 1:

```
#ex[
  Phát biểu nào sau đây đúng/sai?
  #choiceTF(
    [Phương trình  $x + y = 1$  là bậc nhất hai ẩn],
    [Phương trình  $0x + 0y = 1$  là bậc nhất hai ẩn],
    [Phương trình  $x + 0y = 1$  là bậc nhất hai ẩn],
    [Phương trình  $0x + y = 0$  là bậc nhất hai ẩn],
    dapan: "Đ, S, Đ, Đ",
    socot: 2
  )
  #loigiai[
    #itemchoice[
      + Vì thoả  $a \neq 0$ .
      + Vì  $a = b = 0$ .
      + Vì thoả  $a \neq 0$ .
      + Vì thoả  $b \neq 0$ .
    ]
  ]
]
```

Kết quả:

Câu 1. Phát biểu nào sau đây đúng/sai?

- | | |
|--|---|
| a) Phương trình $x + y = 1$ là bậc nhất hai ẩn. | b) Phương trình $0x + 0y = 1$ là bậc nhất hai ẩn. |
| c) Phương trình $x + 0y = 1$ là bậc nhất hai ẩn. | d) Phương trình $0x + y = 0$ là bậc nhất hai ẩn. |

Lời giải.

a) Đúng

Vì thoả $a \neq 0$.

b) Sai

Vì $a = b = 0$.

c) Đúng

Vì thoả $a \neq 0$.

d) Đúng

Vì thoả $b \neq 0$.

Ví dụ minh hoạ 2:

```
#ex[
  Phát biểu nào sau đây đúng/sai?
  #choiceTF(
    [Phương trình  $x + y = 1$  là bậc nhất hai ẩn],
    [Phương trình  $0x + 0y = 1$  là bậc nhất hai ẩn],
    [Phương trình  $x + 0y = 1$  là bậc nhất hai ẩn],
    [Phương trình  $0x + y = 0$  là bậc nhất hai ẩn],
    dapan: "Đ, S, Đ, Đ",
    kieu: "t",
  )
  #loigiaiai[
    #itemchoice(socot: 2)[
      + Vì thoả  $a \neq 0$ .
      + Vì  $a = b = 0$ .
      + Vì thoả  $a \neq 0$ .
      + Vì thoả  $b \neq 0$ .
    ]
  ]
]
```

Kết quả:

Câu 2. Phát biểu nào sau đây đúng/sai?

Phát biểu	Đ	S
a) Phương trình $x + y = 1$ là bậc nhất hai ẩn.		
b) Phương trình $0x + 0y = 1$ là bậc nhất hai ẩn.		
c) Phương trình $x + 0y = 1$ là bậc nhất hai ẩn.		
d) Phương trình $0x + y = 0$ là bậc nhất hai ẩn.		

Lời giải.

a) Đúng

Vì thoả $a \neq 0$.

b) Sai

Vì $a = b = 0$.

c) Đúng

Vì thoả $a \neq 0$.

d) Đúng

Vì thoả $b \neq 0$.

#SHORTANS – CÂU TRẢ LỜI NGẮN

Sử dụng tham số **kieu** để thay đổi kiểu in kết quả:

- **"..."** : hiện 1 dòng chấm ngắn (mặc định).
- **"oly"** : hiện 4 ô vuông.
- **"3"** : một ô chữ nhật dài 3cm.

Lưu ý: **#shortans** cũng tự lưu đáp án vào hệ thống giống **#choice**. chỉ cần đặt bên trong cặp **#BatDauLuuDapAn** và **#KetThucLuuDapAn** là **#XuatDapAn** sẽ tự liệt kê luôn đáp án của các câu **#shortans**, không cần làm gì thêm.

Ví dụ minh họa:

```
#ex[
  Tổng hai nghiệm của phương trình... bằng bao nhiêu?
  #shortans("1234", kieu: "...")
]
#ex[
  Tổng hai nghiệm của phương trình... bằng bao nhiêu?
  #shortans("1234", kieu: "oly")
]
#ex[
  Tổng hai nghiệm của phương trình... bằng bao nhiêu?
  #shortans("1234", kieu: "3")
]
```

Kết quả:

Câu 1. Tổng hai nghiệm của phương trình... bằng bao nhiêu?

Kết quả:

Câu 2. Tổng hai nghiệm của phương trình... bằng bao nhiêu?

Kết quả:

--	--	--	--

Câu 3. Tổng hai nghiệm của phương trình... bằng bao nhiêu?

Kết quả:

--

– LƯU VÀ XUẤT BẢNG ĐÁP ÁN TỰ ĐỘNG

Cách dùng:

```
#BatDauLuuDapAn("<tên bản sao lưu>")
#KetThucLuuDapAn("<tên bản sao lưu>")
#XuatDapAn(
    "<tên bản sao lưu>",
    socot: <số cột>,
    duongthang: <độ rộng đường thẳng> + <màu sắc>,
    botron: <độ bo tròn>,
    khoangcach: <khoảng cách giữa các ô>,
    maunen: <màu nền của ô>,
)
```

Ví dụ minh họa:

```
#BatDauLuuDapAn("9D1-1-Tracnghiem")
#ex[ ... #choice(..., dapan: "B") ... ]
#ex[ ... #choice(..., dapan: "B") ... ]
#counter("ex_count").update(0) // reset bộ đếm
#ex[ ... #choiceTF(..., dapan: "Đ, S, Đ, Đ") ... ]
...
#KetThucLuuDapAn("9D1-1-Tracnghiem")

#XuatDapAn(
    "9D1-1-Tracnghiem",
    socot: 6,
    duongthang: 1pt + blue,
    maunen: blue.lighten(95%)
)
```

Kết quả:

Câu 1. B

Câu 2. B

Câu 1. Đ, S, Đ, Đ

Câu 2. Đ, S, Đ, Đ

Câu 1. 1234

Câu 2. 1234

Câu 3. 1234

Lưu ý: Nếu có reset bộ đếm giữa chừng thì vẫn lưu đáp án theo đánh số của đề bài.

#DAPSO – ĐÁP SỐ VẮN TẮT CHO BÀI TOÁN TỰ LUẬN

Ví dụ minh họa:

```
#bt[
  Giải phương trình  $2x = 6$ .
  #dapso( $x = 3$ , hien: true)
  #loigiaih[Hướng dẫn giải...]
]
```

Kết quả:

Bài 1. Giải phương trình $2x = 6$.

Đáp số: $x = 3$

Lời giải.

Hướng dẫn giải...

Lưu ý:

- Lệnh này sẽ KHÔNG lưu lại kết quả bằng `#BatDauLuuDapAn` và `#KetThucLuuDapAn`.
- Có 2 cách bật/tắt hiển thị đáp số cho **toàn bộ** tài liệu kể từ vị trí đặt lệnh:
 - + Gọi thẳng state: `#dapso_show_state.update(true)` / `update(false)`.
 - + Hoặc dùng lệnh tắt có sẵn: `#showdapso` để hiện và `#hidedapso` (hoặc `#exitdapso`) để ẩn.
- Tham số `hien`: truyền trực tiếp trong `#dapso(...)` luôn được ưu tiên xử lý hơn cấu hình chung bằng các cách trên.

#LISTEX #ITEMIZE #ENUMERATE – TẠO DANH SÁCH

A #listEX – tự động sắp xếp

Lệnh `#listEX` dùng để lập danh sách thay cho `enum`, trong đó các ý của bài tập sẽ tự động dàn cột tương tự bên LaTeX (tối đa 4 cột). Nếu muốn chỉ định số cột cụ thể thì ta thêm tham số `socot`. Cú pháp lệnh đầy đủ như sau:

```
#listEX(socot: <số cột>)[
+ Ý thứ nhất;
+ Ý thứ hai;
+ Ý thứ ba.
]
```

Lưu ý: Nếu số lượng ý nhỏ từ 26 ý trở xuống thì vẫn đánh bằng chữ cái. Ngược lại, nếu trên 26 thì sẽ chuyển sang đánh bằng số.

Ví dụ minh họa:

```
#listEX[
+  $x + 2y = 3$ ;
+  $0x + y = -2$ ;
+  $x + 0y = 3$ .
]
```

Kết quả:

a) $x + 2y = 3$;

b) $0x + y = -2$;

c) $x + 0y = 3$.

B #itemize #enumerate – thay thế cho LaTeX

Hai lệnh này chỉ là bọc lại `list` / `enum` gốc của Typst với khoảng cách trên/dưới và kiểu đánh số quen thuộc bên LaTeX, cú pháp bên trong dùng `-` / `+` như bình thường của Typst:

```
#itemize[
- Ý gạch đầu dòng thứ nhất
- Ý gạch đầu dòng thứ hai
]
```

```
#enumerate[
+ Ý đánh số thứ nhất
]
```

```
+ Ý đánh số thứ hai  
]
```

Kết quả:

- Ý gạch đầu dòng thứ nhất
- Ý gạch đầu dòng thứ hai
- a. Ý đánh số thứ nhất
- b. Ý đánh số thứ hai

Khác biệt với `#listEX` : hai lệnh `#itemize` và `#enumerate` không tự chia cột, chỉ đổi kiểu marker/ đánh số và thêm khoảng cách khối. Khi cần danh sách các ý của một câu hỏi (tự chia cột theo bề rộng trang), dùng `#listEX` .